PES UNIVERSITY

Data Structures

Dilip Maripuri
Associate Professor
Department of Computer Applications

Session Outline

Graphs

- Introduction
- Terminologies
- Applications of Graph

Graphs

- In many cases we encounter problems that are defined in terms of a number of objects or entities that have some relationships among them.
- We then try to answer interesting questions.
  - Campus map
  - Travelling salesperson
  - Circuit layout
  - Project scheduling
  - Oil flow
  - Flight scheduling

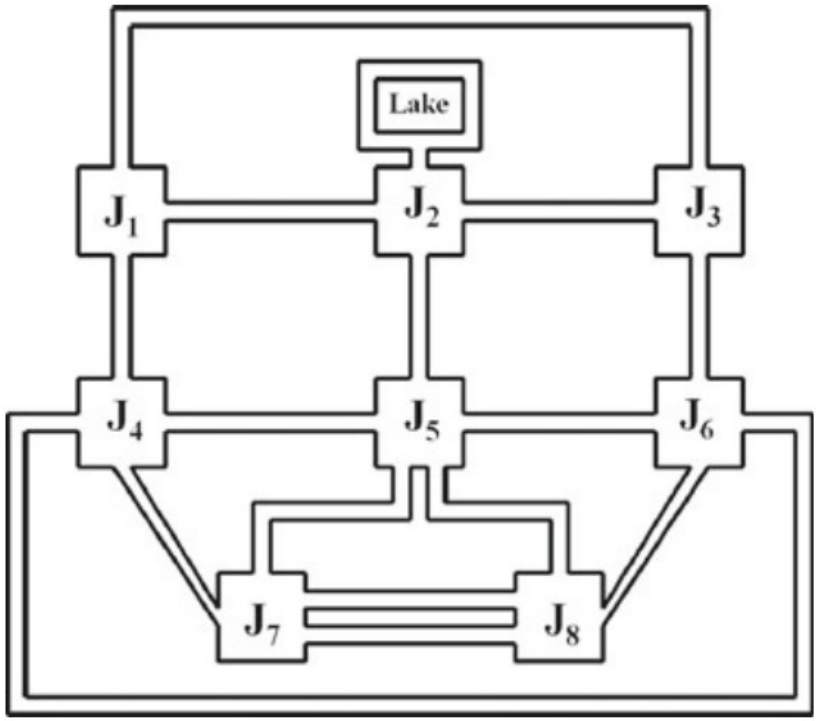
Graphs

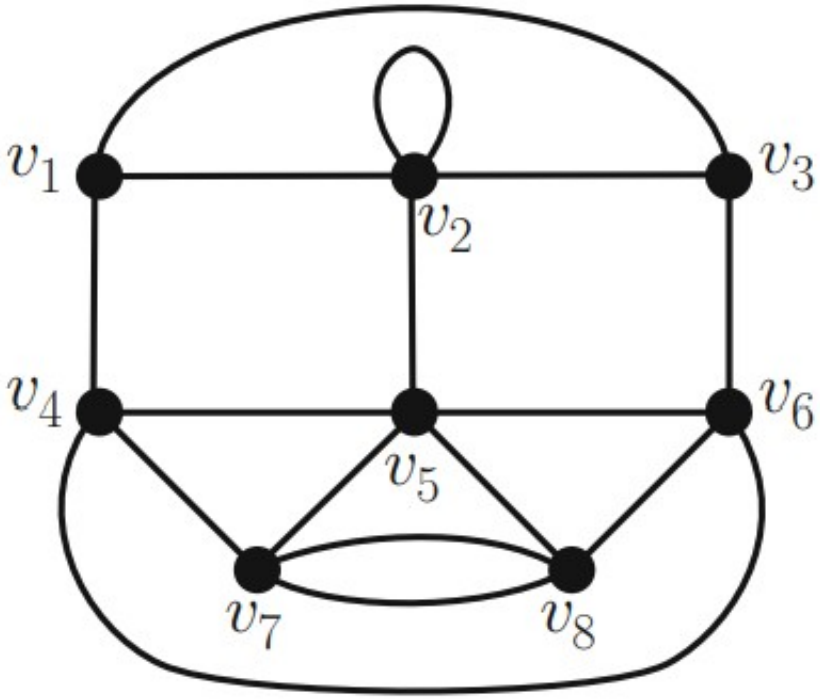
- Graphs are the basic mathematical formulation we use to tackle such problems.
  - Basic definitions and properties.
  - Computer representation.
  - Some applications.

Graphs

- A graph is a mathematical structure used to model pairwise relations between objects.
- A graph is non-linear data structure.
- It consists of two fundamental components:
  - **Vertices (or Nodes)**: These are the points or entities in the graph, representing the objects in the model.
  - **Edges (or Links)**: These are the connections between the vertices, symbolizing the relationships or interactions between the objects.

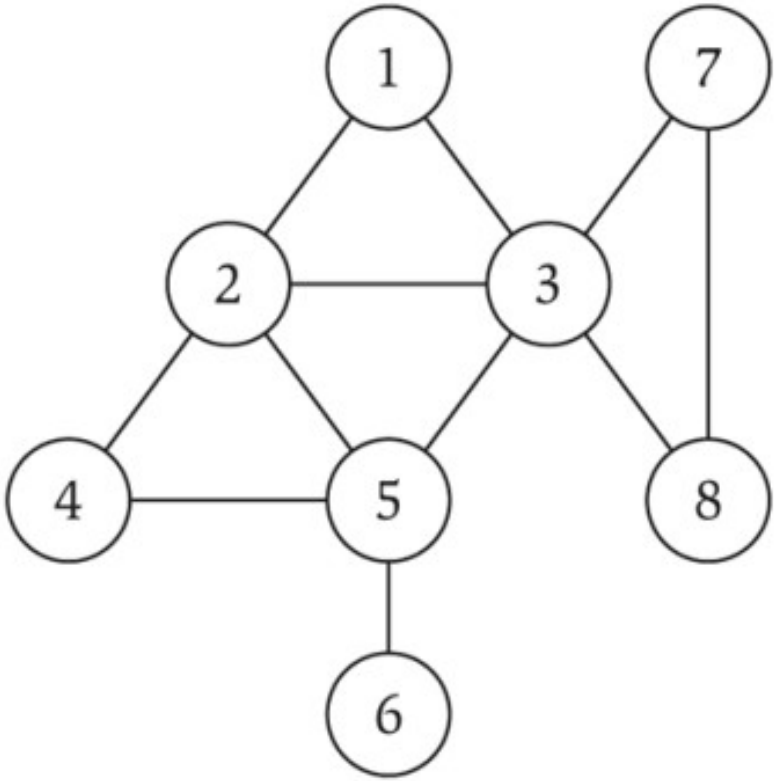
- Consider a road network of a town consisting of streets and street intersections.





Graphs

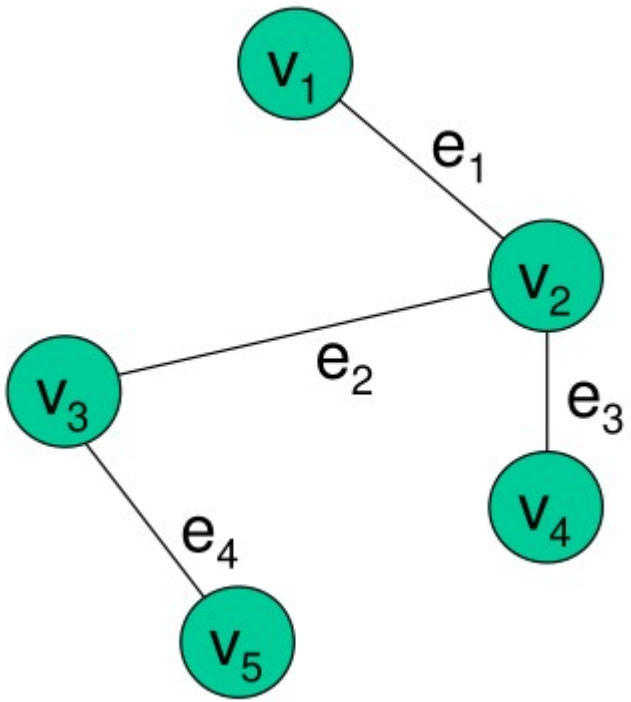
- **Notation :** G = (V, E)
  - V = nodes (or vertices).
  - E = edges (or arcs) between pairs of nodes.
  - Captures pairwise relationship between objects.
  - Graph size parameters: **n = | V |, m = | E |**.
    - The number of Vertices of a Graph G is its order
      - Denoted by **|G|**
    - The number of Edges is denoted by **||G||**

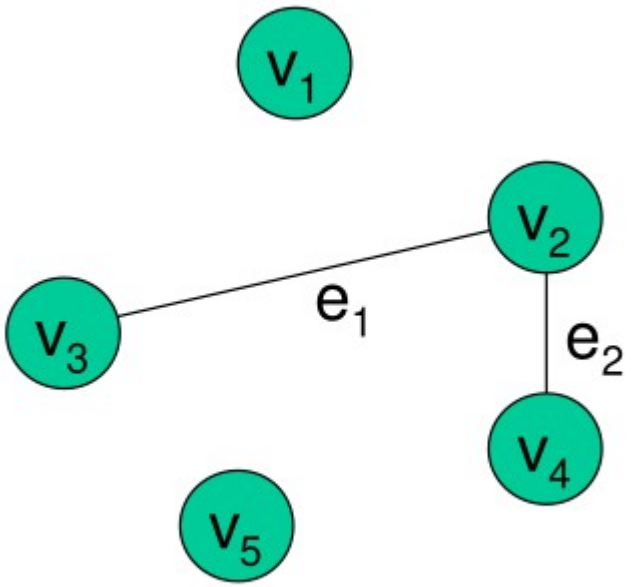


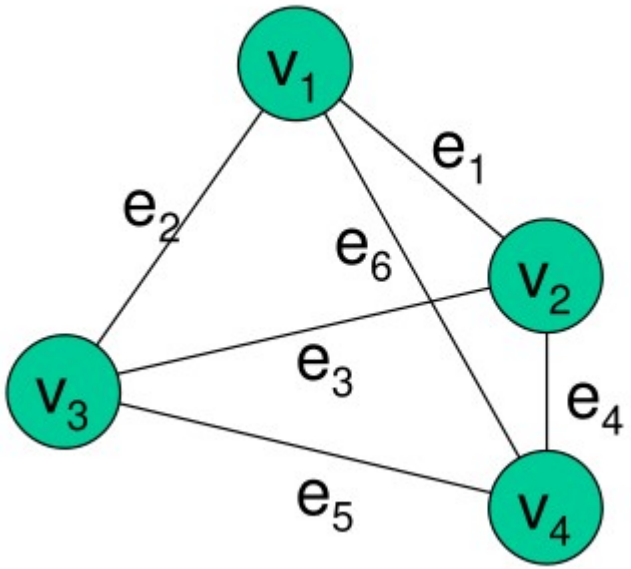
V = { 1, 2, 3, 4, 5, 6, 7, 8 }
E = { 1–2, 1–3, 2–3, 2–4, 2–5, 3–5, 3–7, 3–8, 4–5, 5–6, 7–8 }
m = 11, n = 8

Data Structures

Graphs

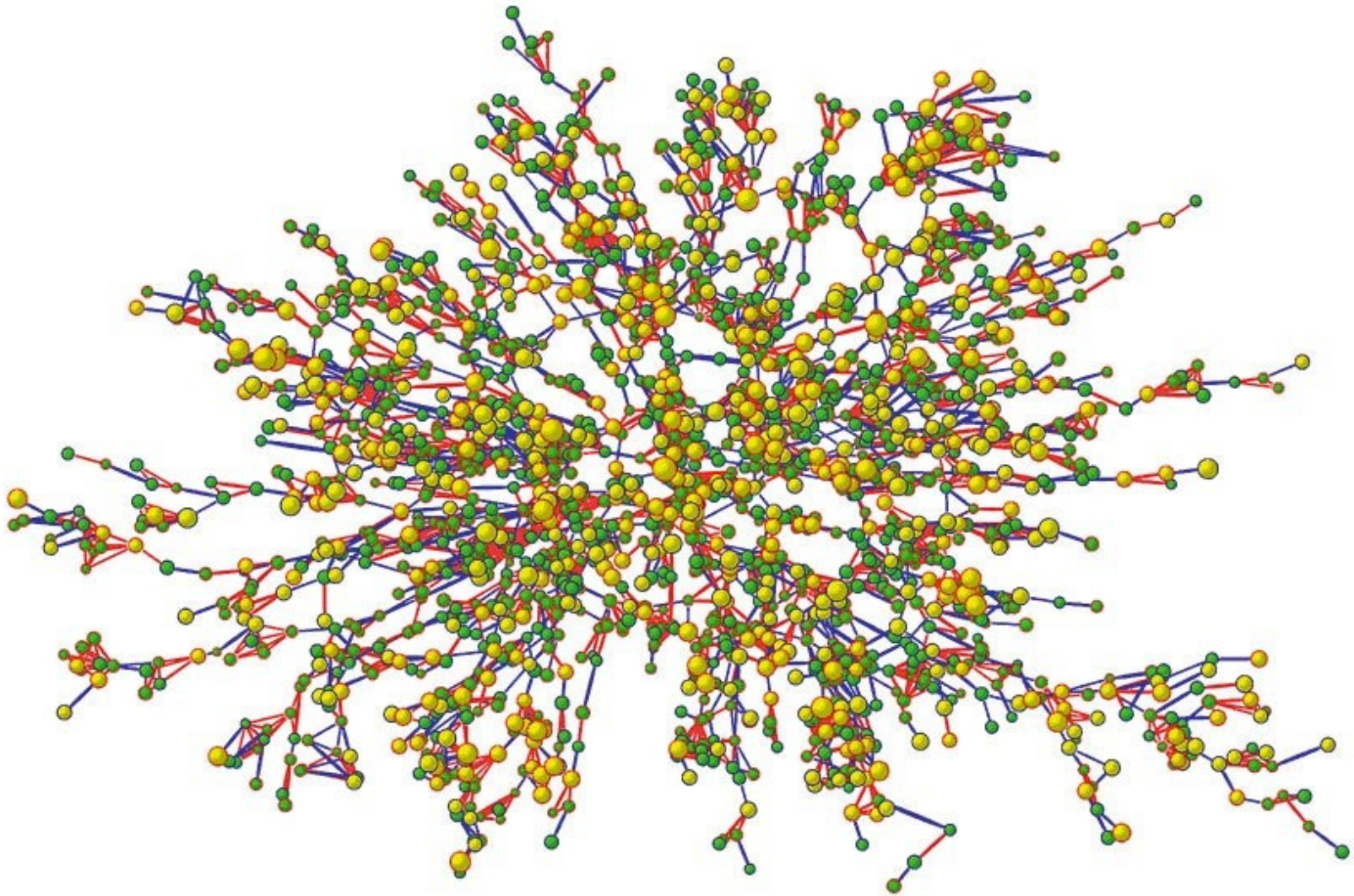






Data Structures

Graphs



- Node represents one person
- Nodes with red borders denote women
- Nodes with blue borders denote men
- The size of each Node is proportional to the person's BMI
- The interior color of the Node indicates the person's obesity status
- The colors of the edge indicates indicate the relationship between persons represented by Node

The Spread of Obesity in a Large Social Network over 32 Years - https://www.nejm.org/doi/full/10.1056/nejmsa066082

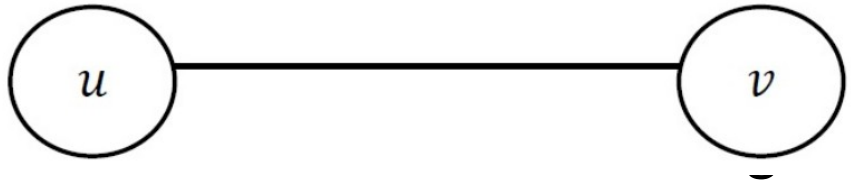
Data Structures

Graphs

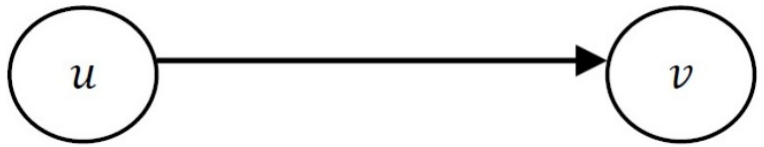
graph	node	edge
communication	telephone,computer	fiber optic cable
circuit	gate,register,processor	wire
mechanical	joint	rod,beam,spring
financial	stock,currency	transactions
transportation	street intersection,airport	highway,airway route
internet	class C network	connection
game	board position	legal move
social relationship	person,actor	friendship,movie cast
neural network	neuron	synapse
protein network	protein	protein-protein interaction
molecule	atom	bond

Graphs – Edges

- **Undirected Edge**
  - An undirected edge is a bidirectional edge. If th between vertices A and B then edge (A , B) is equal to edge (B , A).



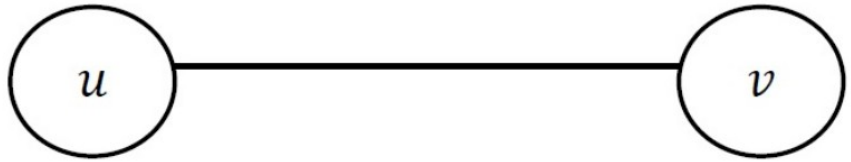
- **Directed Edge**
  - A directed edge is a unidirectional edge. If there is a directed edge between vertices A and B then edge (A , B) is not equal to edge (B , A).



- **Weighted Edge**
  - A weighted edge is an edge with cost on it.

Graphs – Edges

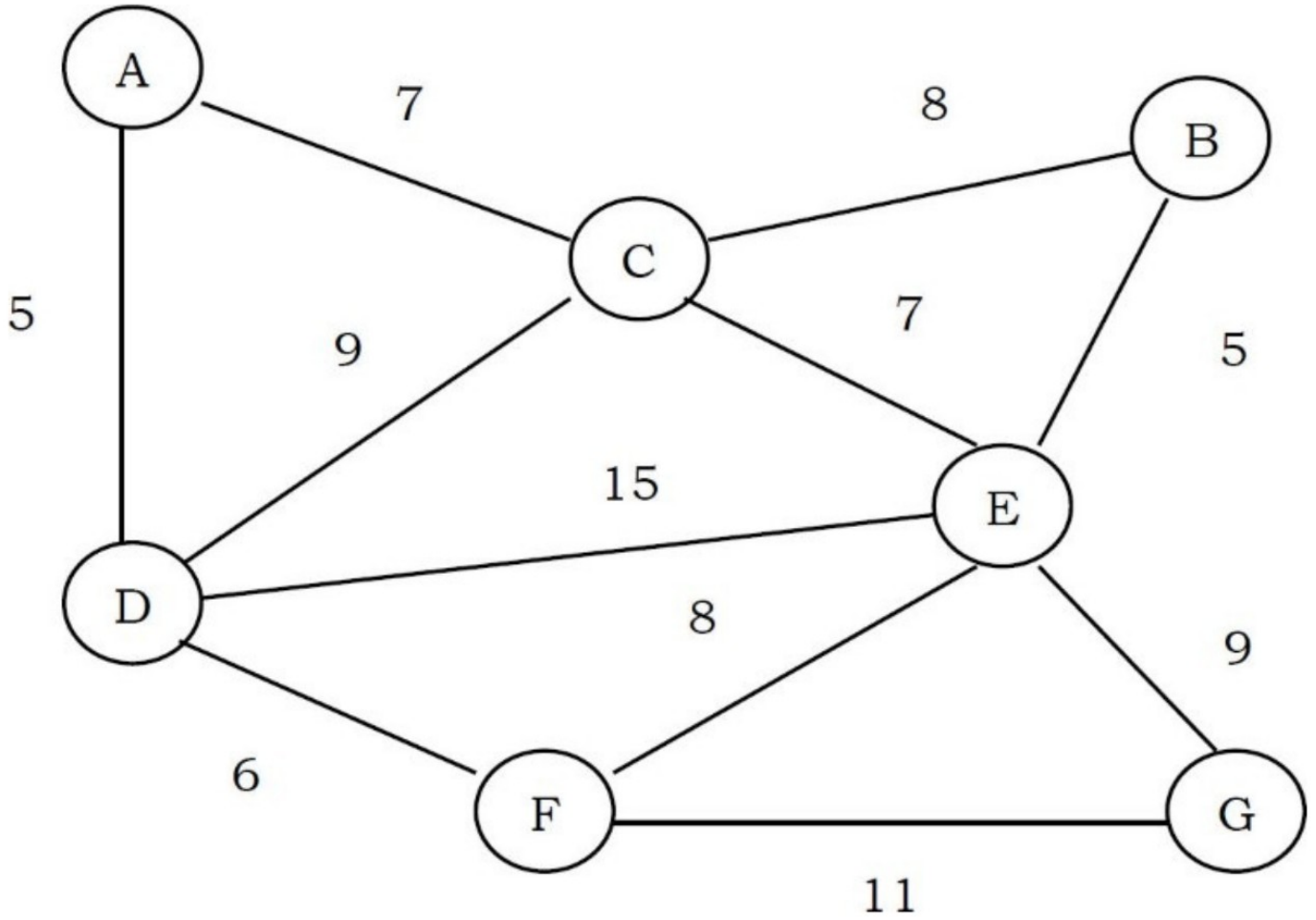
- **Incident Edge**



  - An edge (u,v) is **incident** to both vertex u and vertex v.
  - Vertex u is **adjacent** to vertex v if there is some edge to which both are incident i.e. there is an edge between them.

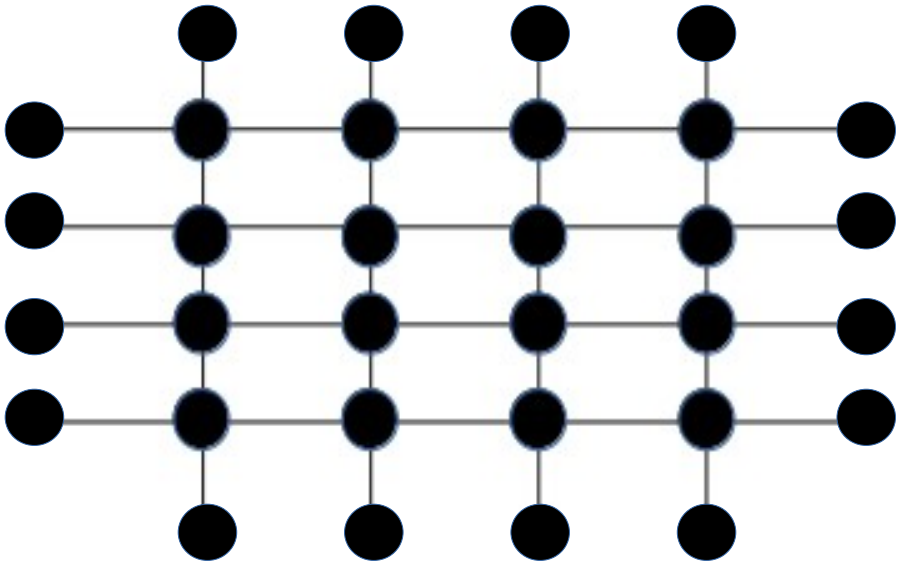
Graphs – Edges

- **Weighted Edge**



Graphs – Types of Graphs

- **Graph**
  - Defined as a pair G = (V, E)
    - V is a set of vertices
    - E is a set of edges
- **Finite Graph**
  - |V| & |E|is finite (V,E have a specific, countable number of elements)
- **Infinite Graph**
  - V,E have have a uncountable number of elements

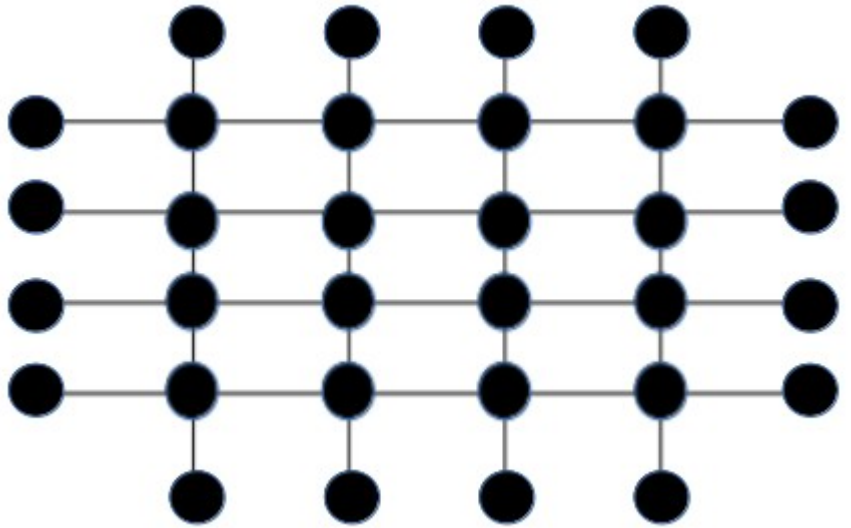


Graphs – Types of Graphs

- **Trivial Graph**
  - Simplest form of Graph
  - Has exactly one vertex and no edges.
    - **|V| = 1 & |E|=0**
  - Unconnected
    - Inherently unconnected since there are no edges to connect the single vertex to anything else.

Graphs – Types of Graphs

- **Simple Graph**
  - **No Parallel Edges**
    - There cannot be more than one edge between any two vertices.
    - For any two vertices **v1, v2 ∈ V** , there exists at most one edge **e ∈ E** that connects **v1** and **v2**.
  - **No Self-loops**
    - Does not allow self-loops.
    - For any vertex **v ∈ V** , there does not exist an edge **(v, v) ∈ E**.
    - This condition explicitly forbids the presence of edges that connect a vertex to itself.



Graphs – Types of Graphs

PES UNIVERSITY

- **Simple Graph**
  - **Undirected simple graph**
    - Each edge is an unordered pair of distinct vertices, meaning (v1, v2) is equivalent to (v2, v1) and both represent the same edge.
  - **Directed simple graph**
    - Each edge is an ordered pair of distinct vertices, meaning (v1, v2) and (v2, v1) are considered different edges if both are present.

A simple graph is typically used to model relationships or connections where the presence of multiple connections between the same entities (parallel edges) or self-referential connections (self-loops) is either not possible or not of interest.

Graphs – Types of Graphs

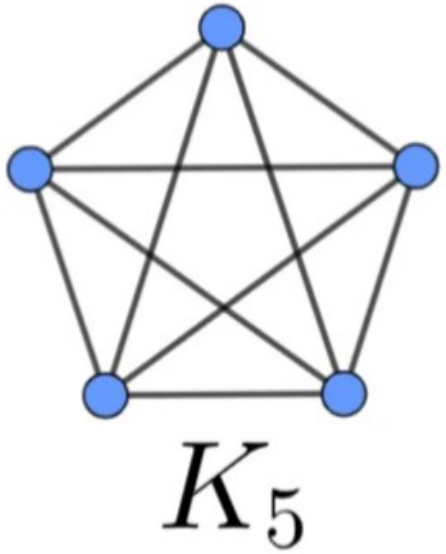
PES UNIVERSITY

- **Multigraph Graph**
  - Allows multiple edges between the same two vertices. These edges are also called parallel edges.
  - Does not allow loops, meaning an edge cannot connect a vertex to itself.

- **Null Graph**
  - A null graph is a graph with no edges. It is defined as a pair G = (V, E), where
    - **|V| is finite & |E| is ∅**
  - **No Connectivity**
    - There are no connections between any pair of vertices.
  - **Variation in Size**
    - The number of vertices can vary, including the possibility of having no vertices at all (an empty graph).

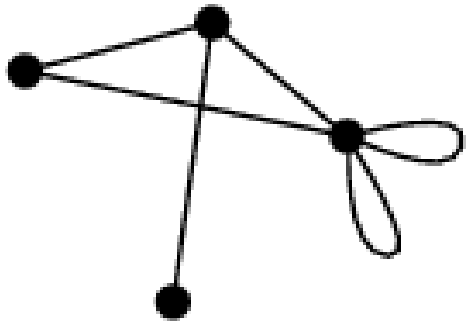
Graphs – Types of Graphs

- **Complete Graph**
  - Is a graph where every pair of distinct vertices is connected by a unique edge. It is also defined as a pair G = (V, E), with:
    - **|V| = n** , where n is a non-negative integer.
    - E is a set of edges such that every possible pair of vertices is connected by exactly one edge. (Maximum Connectivity)
      - The total number of edges in a complete graph with n vertices is given by the formula **[n * (n - 1)] / 2.**
  - A complete graph represents the maximum possible number of edges between vertices in a graph.
    - K_5 indicates a Completely Connected Graph of 5 Vertices



Graphs – Types of Graphs

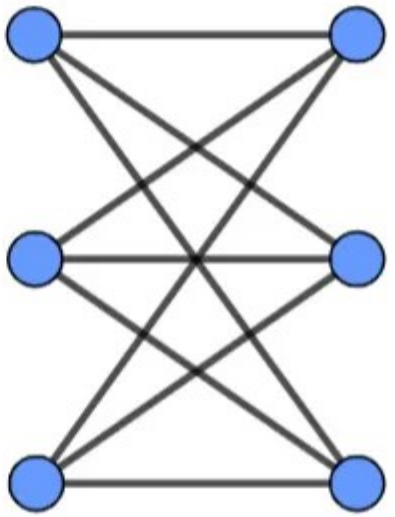
- **Pseudo Graph**
  - A graph that relaxes some of the constraints of a simple graph.
  - Unlike simple graphs, in a pseudo graph, edges can do two things that are not allowed in simple graphs
    - Loop: An edge can connect a vertex to itself.
    - Multiple Edges: There can be multiple edges connecting the same pair of vertices. These are known as parallel edges.



pseudograph

Graphs – Types of Graphs

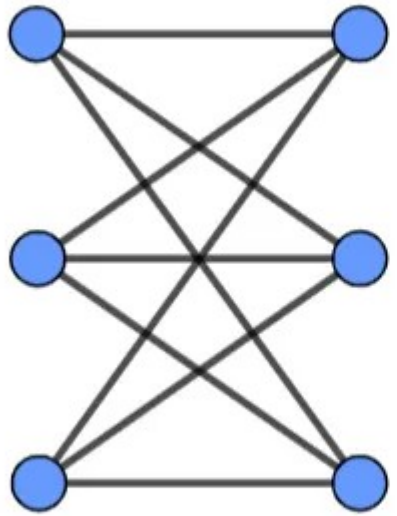
- **Regular Graph**
  - A graph where every vertex has the same number of adjacent vertices, meaning each vertex has the same degree. This can be defined mathematically as follows:
    - For every vertex v in V, the degree of v (denoted as deg(v)), is the same for all vertices.
    - If every vertex has a degree of k, then the graph is specifically called a ”k-regular graph”.

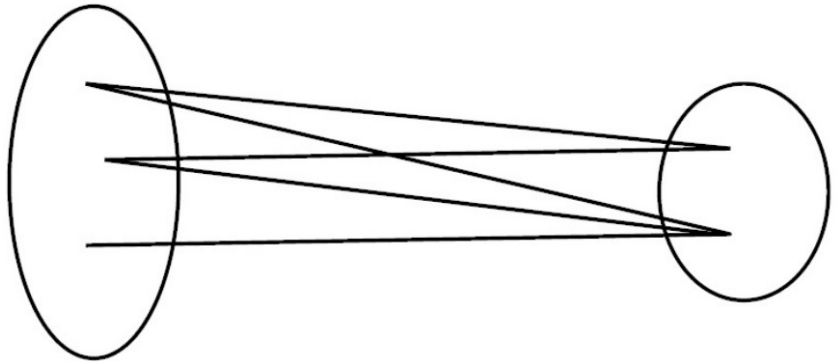


$K_{3,3}$

Graphs – Types of Graphs

- **Bipartite Graph**
  - **Vertices Divided into Two Disjoint Sets**
    - The set of vertices V can be divided into two distinct and disjoint subsets V1 and V2.
  - **Edges Connect Vertices from Different Sets**
    - Every edge in the set of edges E connects a vertex from V1 to a vertex from V2.
    - There are no edges connecting vertices within the same subset.
  - A bipartite graph is denoted as G = (V1 ∪ V2, E)





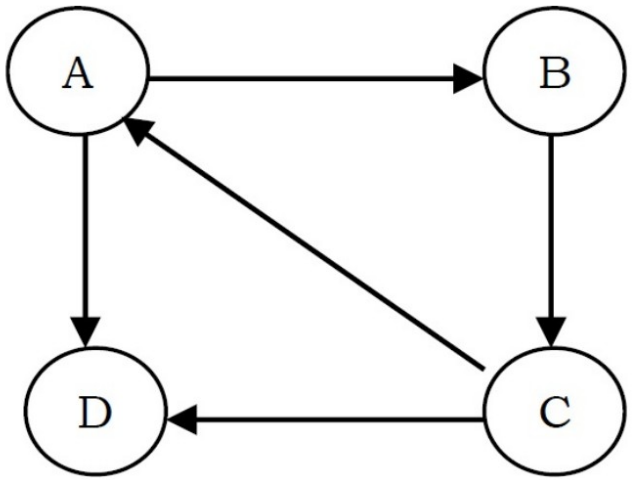
Graphs – Types of Graphs

PES UNIVERSITY

- **Labelled Graph**
  - A graph where each vertex and possibly each edge is assigned a unique label.
  - **Unique Labels for Vertices**
    - Each vertex in the set of vertices V is given a unique identifier or label.
    - If there are n vertices, they could be labeled with numbers 1 through n, or any other unique identifiers.
  - **Possibly Labeled Edges**
    - Similarly, edges in the set of edges E may also be labeled.
    - These labels can represent weights, capacities, distances, or simply unique identifiers for the edges.
  - Labeled graphs are crucial in scenarios where the identity of vertices and edges matters, such as in network analysis, shortest path problems, or when modeling systems where relationships or entities have unique identifiers.

Graphs – Types of Graphs

- **Directed Graph**
  - defined as G = (V, E) where
    - V represents the set of vertices (or nodes) in the graph. V = {v1, v2, . . . , vn} for a finite set, or V can be infinite.
    - E represents the set of directed edges (or arcs) in the graph.
      - E $\subseteq$ {(vi, vj)|vi, vj $\in$ V }, where each edge is anordered pair indicating direction from vertex vi to vertex vj .
    - In a Directed Graph, (vi, vj) != (vj , vi) unless both pairs are explicitly defined as edges in E



Graphs – Types of Graphs

PES UNIVERSITY

- **Connected Graph**
  - In the context of undirected graphs,it is defined as G = (V, E)
    - V is the set of vertices.
    - E is the set of edges. $E \subseteq \{\{vi, vj\} | vi, vj \in V\}$, where each edge is an unordered pair of vertices.
  - A graph is connected if and only if for every pair of vertices vi and vj in V , there exists a path that connects vi to vj .
  - A path is a sequence of edges that connects a sequence of vertices

Graphs – Types of Graphs

- **Disconnected Graph**
  - The graph is disconnected if there exists at least one pair of vertices vi, vj ∈ V for which no path exists between them.
  - There are vertices in the graph that are not reachable from each other.
  - In a Disconnected Graph, the graph can be partitioned into two or more subgraphs, such that no vertices are connected between different subgraphs

Graphs – Types of Graphs

- **Cyclic Graph**
  - A graph that contains at least one graph cycle.
    - **In a Directed Graph**
      - A cycle occurs if there is a path of directed edges that starts at a vertex and returns to the same vertex, following the direction of the edges.
    - **In an Undirected Graph**
      - A cycle occurs if there is a path that starts and ends at the same vertex without repeating any edge.

Graphs – Types of Graphs

- **Acyclic Graph**
  - A graph that contains no cycles.
    - **In a Directed Graph**
      - The graph is called a Directed Acyclic Graph (DAG) if there are no cycles.
      - In a DAG, it is impossible to start at any vertex and follow a sequence of directed edges to return to the same vertex.
    - **In an Undirected Graph**
      - The graph is acyclic if there is no path that starts and ends at the same vertex without reusing an edge.

Graphs – Incidence Arc

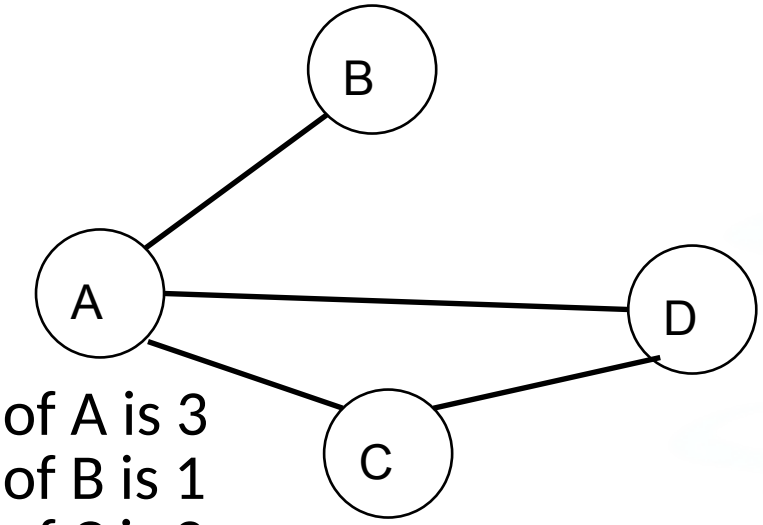
- **Node is incident to an arc**
  - In a digraph, an arc (directed edge) connects two nodes, with an arrow indicating the direction. A node is considered incident to an arc if:
    - The arc originates from the node (called the TAIL) and points to another node.
    - The arc terminates at the node (called the HEAD) and comes from another node.
- In an undirected graph, an edge simply connects two nodes without any direction.
  - A node is considered incident to an edge if it is one of the two connected nodes by the edge.

Graphs – Degree of Vertex

- **Degree of a node**
  - The degree of a node in a graph (both directed and undirected) refers to the total number of edges that are incident to that node.
    - In an undirected graph, the degree considers all edges connected to the node, regardless of direction.
    - In a directed graph, the degree does not make sense as a single value.
      - We use concepts of indegree and outdegree for directed graphs.

Graphs – Degree of Vertex

- **Degree of a vertex**



- Degree of A is 3
- Degree of B is 1
- Degree of C is 2
- Degree of D is 2

Graphs – Degree of Vertex

- **Indegree and Outdegree (directed graphs only)**
  - In a directed graph:
    - **Indegree** of a node refers to the number of incoming arcs that point towards the specific node.
    - **Outdegree** of a node refers to the number of outgoing arcs that originate from the specific node and point to other nodes.

Graphs – Sparse Graph

- **Sparse Graph**
  - A sparse graph has relatively few edges compared to the maximum possible number of edges for a given number of vertices.
  - There are not many connections between the vertices.
  - The number of edges in a sparse graph is in the order of the number of vertices, meaning it scales linearly with the size of the graph.
  - **Examples**
    - A social network where most people only have connections with a small number of others.
    - A road network in a rural area with few connecting roads.

Graphs – Dense Graph

- **Dense Graph**
  - A dense graph has a large number of edges, close to the maximum possible number of edges for a given number of vertices.
  - This means most vertices are connected to many other vertices.
  - The number of edges in a dense graph is often close to the square of the number of vertices, indicating a quadratic growth rate with size.
  - Examples
    - A social network where most people are connected to a large number of others (e.g., actors in Hollywood).
    - A road network in a densely populated city with many intersecting roads.

Graphs – Dense Graph

Feature	Sparse Graph	Dense Graph
Number of Edges	Few (linear with vertices)	Many (close to square of vertices)
Connectivity	Fewer connections between vertices	Most vertices connected to many others
Example	Social network (limited connections)	Road network (densely connected)

Graphs – Where do we find usage ?

PES UNIVERSITY

- **Algorithm Design and Analysis**
  - Optimization problems, such as finding the shortest path, the minimal spanning tree, and network flow problems
  - DFS, BFS, Dijkstra's algorithm
- **Data Structures**
  - Trees, a type of graph, are foundational in designing and understanding data structures like binary search trees, heaps, and trie.
- **Network Analysis:**
  - Graphs represent networks such as social networks, computer networks, and telecommunication networks, where nodes represent entities and edges represent connections or interactions.
  - Algorithms for network analysis include finding the shortest path, network flow, and detecting communities within networks.

Graphs – Where do we find usage ?

- **Operating Systems**
  - Resource allocation in operating systems can be modeled using graphs.
  - Deadlock detection algorithms, for instance, often use a special type of graph called a resource allocation graph.
- **Routing and Navigation**
  - GPS and routing software use graph algorithms to find the best route from one location to another over a network of roads or pathways.

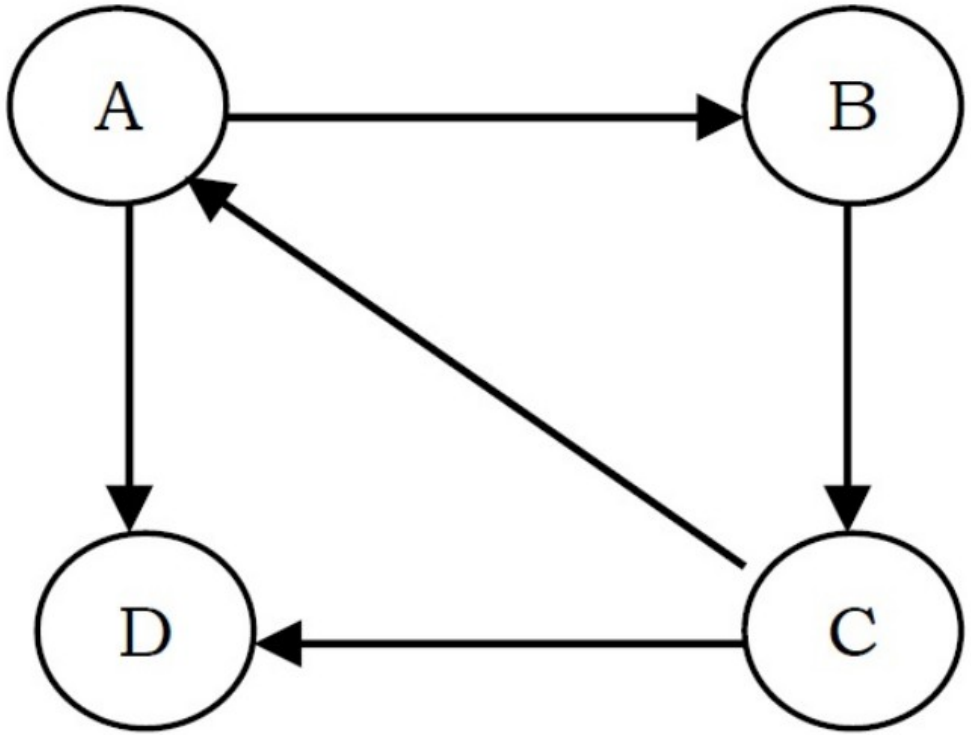
Graphs – Representation

- To manipulate graphs we need to represent them in some useful form.
- There are three ways of representing the graphs
  - Adjacency Matrix
  - Adjacency List
  - Adjacency Set

Graphs – Representation – Adjacency Matrix

- An adjacency matrix is a 2D array where the rows and columns represent the vertices in the graph.
  - Each cell (i, j) in the matrix indicates whether there's an edge from vertex i to vertex j.
  - If there is an edge, the cell contains 1 ,else it contains 0.
- For weighted graphs, the cell can contain the weight of the edge.
  - This representation is simple and it's easy to determine whether an edge exists between two vertices.
  - It can be space-inefficient for sparse graphs (graphs with far fewer edges than the maximum possible), as it requires space for every possible edge, whether it exists or not.

Graphs – Representation – Adjacency Matrix



	A	B	C	D
A	0	1	0	1
B	0	0	1	0
C	1	0	0	1
D	0	0	0	0

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

- **Graph ADT**
  - **Data Structure**
    - Graph
      - numVertices: Represents the number of vertices in the graph
      - adjMatrix: A 2D array representing the adjacency matrix of the graph.
  - **Operations**
    - createGraph(vertices)
      - Description: Initializes a new graph with a specified number of vertices.
      - Inputs: vertices - the number of vertices in the graph.
      - Outputs: A new Graph object.
      - Postconditions: The graph is initialized with no edges (all values in the adjacency matrix are 0).

Graphs – ADT (Adjacency Matrix Mechanism)

- **Graph ADT**
  - **Operations**
    - addEdge(Graph, src, dest)
      - Description: Adds an edge between two vertices in the graph.
      - Inputs:
        - The graph to which the edge is being added.
        - The source vertex and destination vertex.
      - Postconditions:
        - An edge is added from src to dest.
        - If the graph is undirected, an edge is also added from dest to src.

Graphs – ADT (Adjacency Matrix Mechanism)

- **Graph ADT**
  - **Operations**
    - printGraph(Graph)
      - Description: Prints the adjacency matrix of the graph.
      - Inputs: graph the graph to be printed.
      - Outputs: None (prints to console).
      - Postconditions: The adjacency matrix of the graph is printed.
    - freeGraph(Graph)
      - Description: Deallocates memory used by the graph.
      - Inputs: The graph to be deallocated.
      - Postconditions: All memory allocated to the graph is freed.

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

```
Define Structure Graph
    Set numVertices to Integer
    Set adjMatrix to 2D Integer Array Pointer

Function createGraph(vertices)
    Allocate memory for a Graph structure, name it 'graph'
    Set graph's numVertices to vertices

    Allocate memory for graph's adjMatrix as a 2D array
    For each row i from 0 to vertices-1
        Allocate memory for row i of adjMatrix
        For each column j from 0 to vertices-1
            Set graph's adjMatrix at [i][j] to 0

    Return graph
```

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

```
Function addEdge(graph, src, dest)
    Set graph's adjMatrix at [src][dest] to 1
    Set graph's adjMatrix at [dest][src] to 1 (if graph is undirected)
```

```
Function printGraph(graph)
    For each row i from 0 to graph's numVertices-1
        For each column j from 0 to graph's numVertices-1
            Print graph's adjMatrix at [i][j]
        Print newline
```

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

```
Function freeGraph(graph)
    For each row i from 0 to graph's numVertices-1
        Free memory of row i of adjMatrix
    Free memory of adjMatrix
    Free memory of graph
```

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

```
Main Program
    Set numVertices to desired number of vertices
    Create a graph using createGraph with numVertices

    Add edges to the graph using addEdge

    Print the graph using printGraph

    Free the graph using freeGraph
```

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

```
Graph createGraph(int vertices) {
    Graph graph = malloc(sizeof(*graph));
    graph->numVertices = vertices;

    graph->adjMatrix = malloc(vertices * sizeof(int *));
    for (int i = 0; i < vertices; i++) {
        graph->adjMatrix[i] = malloc(vertices * sizeof(int));
        for (int j = 0; j < vertices; j++) {
            graph->adjMatrix[i][j] = 0;
        }
    }

    return graph;
}
```

```
typedef struct {
    int numVertices;
    int **adjMatrix;
} *Graph;
```

Graphs – Representation – Adjacency Matrix Pros & Cons

- **Note:**
  - In C, explicit typecasting of the return value of malloc is not required when assigning to a pointer type.
  - This is because malloc returns a void* type, which in C can be implicitly converted to any other pointer type without a cast.
  - **Graph graph = malloc(sizeof(*graph));**
    - malloc allocates memory of size sizeof(*graph) and returns a void*.
    - This void* is automatically and safely converted to Graph (which is a pointer type).
  - **graph->adjMatrix = malloc(vertices * sizeof(int *));**
    - malloc returns a void* which is automatically converted to int** (since adjMatrix is of type int**).

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

```
void addEdge(Graph graph, int src, int dest) {
    graph->adjMatrix[src][dest] = 1;
    graph->adjMatrix[dest][src] = 1;
}

void printGraph(Graph graph) {
    for (int i = 0; i < graph->numVertices; i++) {
        for (int j = 0; j < graph->numVertices; j++) {
            printf("%d ", graph->adjMatrix[i][j]);
        }
        printf("\n");
    }
}
```

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

```
void freeGraph(Graph graph) {
    for (int i = 0; i < graph->numVertices; i++) {
        free(graph->adjMatrix[i]);
    }
    free(graph->adjMatrix);
    free(graph);
}
```

Graphs – ADT (Adjacency Matrix Mechanism)

PES UNIVERSITY

```c
int main() {
    int numVertices = 4;
    Graph graph = createGraph(numVertices);
    addEdge(graph, 0, 1);
    printGraph(graph);
    freeGraph(graph);
    return 0;
}
```

Graphs – Representation – Adjacency Matrix Pros & Cons

- **Pros of Adjacency Matrix**
  - **Simple Representation**
    - It provides a simple, intuitive way to represent graphs. We can easily visualize the graph's connections and properties.
  - **Edge Access**
    - Checking if an edge exists between two vertices is very efficient – O(1) complexity.
    - This is because we just need to check the value at the corresponding matrix cell.
  - **Edge Addition/Removal**
    - Adding or removing an edge is also O(1), as it involves updating a single element in the matrix.
  - **Support for Weighted Graphs**
    - It can easily represent weighted graphs by storing weights instead of boolean values.

Graphs – Representation – Adjacency Matrix Pros & Cons

- **Cons of Adjacency Matrix**
  - **Space Inefficiency**
    - Requires O(V^2) space, where V is the number of vertices.
    - This is inefficient for sparse graphs
  - **Iterating Over Edges**
    - If we need to iterate over edges, it's inefficient, especially for sparse graphs.
    - The complexity is O(V^2), regardless of the number of edges.
  - **Adding/Removing Vertices**
    - Adding or removing vertices is expensive.
    - It requires creating a new matrix or resizing the existing one, with a complexity of O(V^2).
  - **No Direct Support for Nonexistent Edges**
    - In an unweighted graph, we typically use 0 to represent the absence of an edge, which can be ambiguous if 0 is a valid edge weight in a weighted graph.

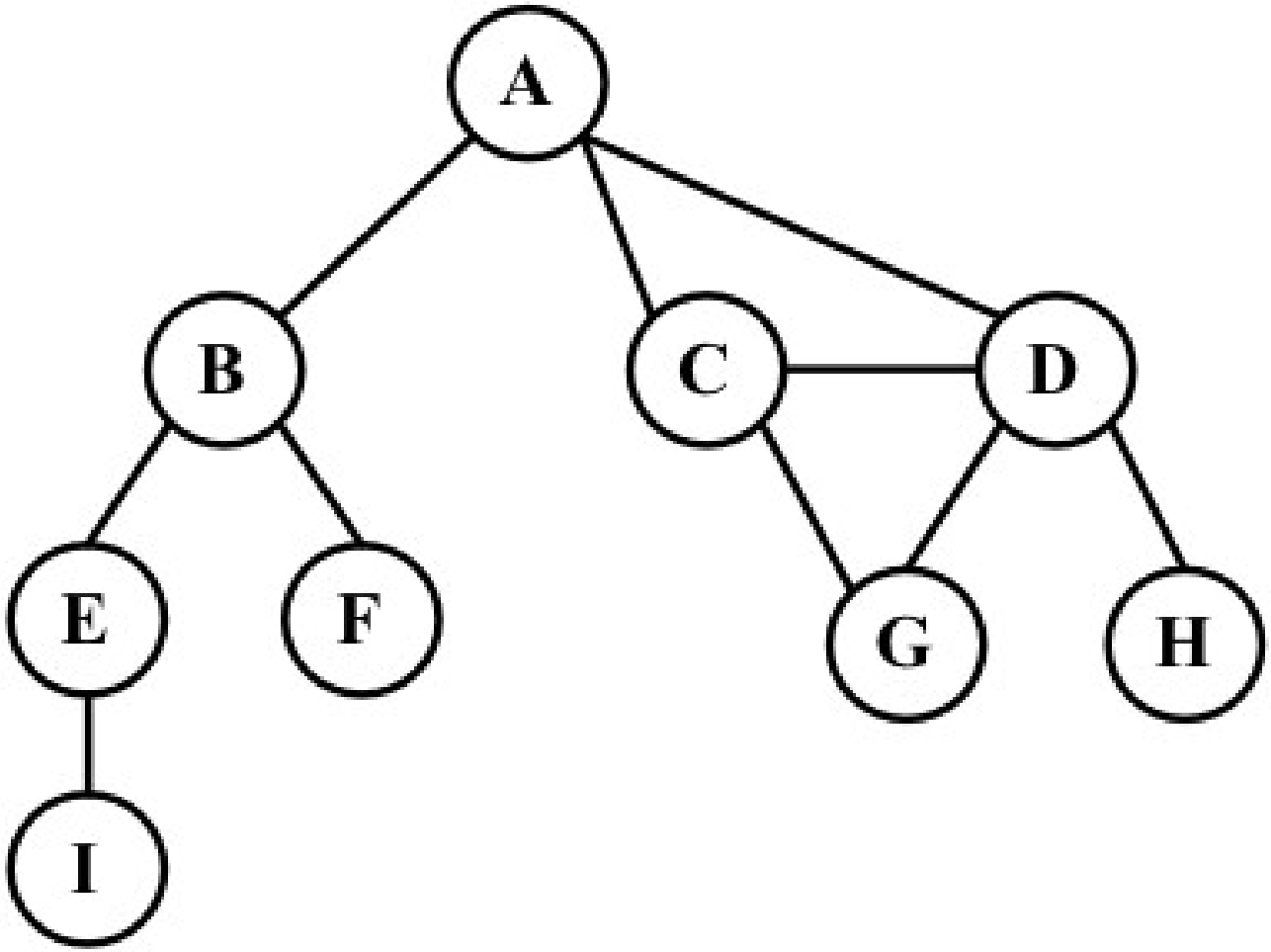
PES UNIVERSITY

- **Space Complexity:** $O(V^2)$, where V is the number of vertices.
- **Time Complexity:**
  - Check if an edge exists: O(1).
  - Add an edge: O(1).
  - Remove an edge: O(1).
  - Iterate over all edges: $O(V^2)$, as we have to go through the entire matrix.
  - Add a vertex: $O(V^2)$ due to the need to create a new matrix.
  - Remove a vertex: $O(V^2)$, as it generally involves creating a new matrix with one less row and column.

Graphs – Representation – Adjacency List

- An adjacency list represents a graph as an array of lists.
- Each element of the array (corresponding to a vertex) contains a list of its adjacent vertices (i.e., vertices that are connected by an edge).
- This representation is more space-efficient for sparse graphs, as it only stores edges that actually exist.
- It's also efficient for iterating over the edges, but can be less efficient for checking the existence of a specific edge, compared to an adjacency matrix.

Graphs – Representation – Adjacency List



A	B	C	D	
B	A	E	F	
C	A	D	G	
D	A	C	G	H
E	B	I		
F	B			
G	C	D		
H	D			
I	E			

Graphs – Representation – Adjacency List

PES UNIVERSITY

```
typedef struct AdjNode {
    int dest;
    struct AdjNode* next;
} *NODE;

typedef struct Adj {
    NODE head;
} *LIST;

typedef struct myGraph {
    int numVertices;
    LIST array;
} *Graph;
```

Graphs – Representation – Adjacency List

PES UNIVERSITY

```
NODE newAdjNode(int dest) {
    NODE newNode = (NODE) malloc(sizeof(struct AdjNode));
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}
```

Graphs – Representation – Adjacency List

PES UNIVERSITY

```
Graph createGraph(int V) {
    Graph graph = (Graph) malloc(sizeof(struct myGraph));
    graph->numVertices = V;

    graph->array = (LIST) malloc(V * sizeof(struct Adj));

    for (int i = 0; i < V; ++i)
        graph->array[i].head = NULL;

    return graph;
}
```

Graphs – Representation – Adjacency List

PES UNIVERSITY

```
void addEdge(Graph graph, int src, int dest) {

    NODE newNode = newAdjNode(dest);
    newNode->next = graph->array[src].head;
    graph->array[src].head = newNode;

    newNode = newAdjNode(src);
    newNode->next = graph->array[dest].head;
    graph->array[dest].head = newNode;
}
```

Graphs – Representation – Adjacency List

PES UNIVERSITY

```
void printGraph(Graph graph) {
    for (int v = 0; v < graph->numVertices; ++v) {
        NODE Curr_Node = graph->array[v].head;
        printf("\nAdjacency list of vertex %d\n head ", v);
        while (Curr_Node) {
            printf("-> %d", Curr_Node->dest);
            Curr_Node = Curr_Node->next;
        }
        printf("\n");
    }
}
```

Graphs – Representation – Adjacency List

PES UNIVERSITY

```
void freeGraph(Graph graph) {
    NODE head,temp;
    if (graph) {
        if (graph->array) {
            for (int v = 0; v < graph->numVertices; ++v) {
                head = graph->array[v].head;
                temp = NULL;
                while (head) {
                    temp = head;
                    head = head->next;
                    free(temp);                }
            }
            free(graph->array);
        }
        free(graph);
    }
}
```

Graphs – Representation – Adjacency List

PES UNIVERSITY

```
int main() {
    int V = 3;
    Graph graph = createGraph(V);
    addEdge(graph, 0, 1);
    addEdge(graph, 1, 2);
    printGraph(graph);
    freeGraph(graph);
    return 0;
}
```

Graphs – Representation – Adjacency List

- **Space Efficiency**
  - It is efficient for sparse graphs as it only stores the vertices that are connected.
- **Performance**
  - Adding an Edge
    - Typically O(1), just adding to the list.
  - Checking for Adjacency
    - Can take O(V) in the worst case for a vertex with degree V, as we might have to traverse the entire list.
  - Removing an Edge
    - Also O(V) in the worst case, as finding the edge to remove may require traversing the list.

Graphs – Representation – Adjacency Matrix vs Adjacency List

PES UNIVERSITY

Feature	Adjacency Matrix	Adjacency List
Structure	A 2D array where each cell at (i, j) indicates whether there is an edge from vertex i to vertex j.	A list (or array) of lists. Each list corresponds to a vertex and contains the vertices adjacent to it.
Space Complexity	$O(V^2)$ where V is the number of vertices. Even if there are few edges, space for all possible edges is allocated.	O(V + E) where V is the number of vertices and E is the number of edges. Space is allocated for each edge.
	O(1) - Directly access the matrix cell to	O(degree(V)) - Need to traverse the list

Graphs – Representation – Adjacency Matrix vs Adjacency List

PES UNIVERSITY

Feature	Adjacency Matrix	Adjacency List
Add/Remove Edge	O(1) - Setting or unsetting the value in the matrix cell.	O(1) for adding (at the beginning of the list) and O(degree(V)) for removal.
Add/Remove Vertex	$O(V^2)$ - Requires creating a new matrix.	O(V) - Requires updating each list for vertex removal; adding a new list for vertex addition.
Memory Usage	High, especially for sparse graphs.	Efficient for sparse graphs.

Graphs – Representation – Adjacency Matrix vs Adjacency List

PES UNIVERSITY

Feature	Adjacency Matrix	Adjacency List
Iterating over Edges	$O(V^2)$ - Need to check every cell in the matrix.	O(E) - Directly iterate over all edges.
Suitability	Preferred for dense graphs, or when constant-time edge query is crucial.	Preferred for sparse graphs, or when space efficiency is crucial and edges are frequently added/removed.
Example	[[0, 1, 0], [1, 0, 1], [0, 1, 0]] for a graph with vertices A, B, C and edges AB, BC.	[[B], [A, C], [B]] for the same graph.

Graphs – Representation – Adjacency Sets

- In an adjacency set, each vertex of the graph has an associated set (typically implemented as a hash set or a tree set in programming) that contains all the vertices that are adjacent to it.
- **Advantages**
  - **Space Efficiency:** Like adjacency lists, adjacency sets are more space-efficient than adjacency matrices for sparse graphs.
  - **Fast Checks for Adjacency:** Checking if two vertices are adjacent can be very fast, especially if a hash set is used, as the average time complexity for checking the presence of an element in a hash set is O(1).
  - **Flexible Representation:** This method can represent both directed and undirected graphs.

Data Structures

Graphs – Representation – Adjacency Sets

- **Structure**
  - In an adjacency set, each vertex has an associated set (such as a hash set or tree set) containing all the vertices that are adjacent to it.
- **Space Efficiency**
  - Similar to adjacency lists, it is space-efficient for sparse graphs.
- **Performance**
  - **Adding an Edge:** Typically O(1) for a hash set or O(log V) for a tree set.
  - **Checking for Adjacency:** O(1) in a hash set and O(log V) in a tree set, significantly faster than in a list, especially for vertices with high degree.
  - **Removing an Edge:** Similar to adding an edge, O(1) for a hash set or O(log V) for a tree set.
- **Use Cases:** Preferred when we need fast checks for the existence of an edge between two vertices.

THANK YOU

Dilip Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu